

Memory-Egress Cryptographic Interlock: A Hardware-Enforced Capability-Separation Model for AI Memory Security

Gated-Egress Safety, Epoch Non-Resurrection, and Noninterference Modulo Authorized Release

Thor Thor

Independent Open-Source Researcher, THOR-SEC

ORCID: 0009-0001-6573-385X codethor@gmail.com

GitHub: <https://github.com/codethor0> <https://codethor0.github.io/thor-sec/>

This research was conducted independently on the author’s own time and is not sponsored by, affiliated with, or representative of any employer.

Preprint v1.0.0, October 2026. First shared publicly on 10 August 2026. Released openly for community review.

Abstract

AI agents increasingly combine persistent memory with network, tool, file, and device capabilities. A compromised or manipulated agent can then hold readable protected state and a path through which that state can leave the trust boundary at the same time, and no instruction-level control removes that combination. This paper develops the Memory-Egress Cryptographic Interlock (MECI), a capability-separation pattern intended for hardware enforcement that makes unreleased protected information unreachable from every enabled gated egress channel. MECI formalizes high-to-egress reachability as reflexive-transitive closure over a state-dependent influence graph, gives an explicit labeling rule for sealed objects, defines a generation-checked, linearizable SafeOpen operation evaluated on the post-commit configuration, and separates gated egress from ambient observations such as timing, access patterns, and memory-bus traffic. We prove gated-egress safety from two local implementation obligations, show that a finite hazard projection refines the graph invariant under a stated soundness condition, prove epoch non-resurrection, give a counterexample showing that memory-egress mutual exclusion does not imply noninterference, and prove noninterference modulo an explicit authorized-release function in which the MECI invariant, rather than an assumed output property, carries the gated-output step. The finite safety core is specified in TLA+ and checked exhaustively within stated bounds by TLC and by an independent dependency-free reference checker. The two agree exactly on reachable-state counts for all five modeled designs, find no violation in the atomic and generation-checked designs, and return shortest counterexamples for three deliberately weakened designs. The paper also gives an Arm CCA/RMM refinement path, a GPU quiescence contract, a representative declassification policy for an AI incident-triage agent, and a comparison with CHERI, seL4, and WASI. No current confidential-computing platform is claimed to implement MECI; the work is a formal specification and falsifiable research program.

Keywords: AI agents; memory security; egress control; confidential computing; noninterference; declassification; capability security; trusted execution environments; formal methods; TLA+

Open-source research statement. This is independent open-source research derived from publicly accessible vendor documentation, published or openly available research, and reproducible formal artifacts developed for this paper. The model, reference checker, captured outputs, and TLA+ specification are released with the paper. The abstract theorems are proved under stated assumptions; platform-level guarantees require refinement and implementation evidence. No claim of peer review, production deployment, or measured end-to-end hardware performance is made.

1. Introduction

AI agents are moving from stateless text generation toward systems that remember prior interactions, retrieve user-specific context, invoke third-party tools, execute code, and communicate with external services. The resulting attack surface differs qualitatively from that of a conventional application. Memory and authority are assembled dynamically inside a planner loop, while the same process may call browsers, APIs, databases, file systems, or Model Context Protocol (MCP) servers. Recent formal work on agentic information flow treats the planner and tool loop as a security boundary that requires explicit semantics rather than prompt-level trust [1], and recent MCP studies show that tool metadata alone can steer an agent toward high-privilege tools [2, 3].

The MECI proposal starts from a deliberately simple question: should a machine ever allow protected memory to be readable while an externally observable egress capability is simultaneously available? In its first form, external egress was represented by E_t , protected-memory decryptability by D_t , and readable secret-dependent plaintext by P_t . The invariant was

$$\Phi_t = \neg(E_t \wedge (D_t \vee P_t)), \quad (1)$$

or equivalently $E_t = 1 \Rightarrow D_t = P_t = 0$. The intended transition order was to revoke decryption, purge secret-dependent plaintext, and only then enable egress.

That formulation captures an important design instinct, but it is not a sufficient security theorem for a real AI system. It treats decryption as a single bit, ignores secret-derived buffers and microarchitectural state, conflates all observations with the network interface, and assumes that deleting readable state makes future output independent of a prior secret. This paper keeps the capability-separation idea and tightens the claim to what can be proved and checked.

1.1 Contributions

1. A state-dependent *influence-graph reachability invariant* over gated egress channels and unreleased protected objects, using reflexive-transitive closure, with an explicit labeling rule for sealed (encrypted-at-rest) objects.
2. A separation of *gated channels*, which a reference monitor can disable, from *ambient observation channels*, which require separate noninterference, microarchitectural, cryptographic, or physical assumptions.
3. A generation-checked SafeOpen control block whose final check is evaluated on the post-commit configuration at an explicit linearization point, and a proof of the stale-snapshot failure that occurs without generation validation.
4. Proofs of gated-egress safety from two local obligations, of refinement from a finite hazard projection to the graph invariant, and of epoch non-resurrection, followed by a counterexample showing that these safety results alone do not imply noninterference.
5. An explicit release function for a representative AI incident-triage agent and a conditional noninterference theorem in which a low-determinism lemma derived from the MECI invariant carries the gated-output step.
6. A TLA+ specification of the finite safety core, checked by TLC and by an independent reference checker that agree exactly on reachable-state counts, with negative controls for stale permits, incomplete guards, export-buffer leakage, release-policy widening, and epoch rollback.
7. A concrete Arm CCA/RMM refinement path and a GPU quiescence contract that distinguish present vendor guarantees from proposed MECI mechanisms.

Table 1: *MECI compared with established capability-security substrates.*

System	Core capability property	Contribution to MECI	Distinction from MECI
CHERI	Unforgeable tagged capabilities with bounds, permissions, provenance, and monotonic restriction in the ISA [8].	Makes memory and object authority, and some reachability edges, concrete and hardware-enforced.	No epoch-bound temporal rule that egress authority and unreleased high reachability are mutually exclusive.
seL4	Capability-based microkernel with machine-checked functional and information-flow proofs for supported configurations [9, 10].	Strong reference-monitor substrate and precedent for proving confidentiality over a capability graph.	MECI adds a cryptographic epoch lifecycle and a cross-device SafeOpen cut; device and IOMMU assumptions remain configuration-specific.
WASI	Link-time imports and unforgeable runtime handles grant access to host resources [11].	Application and tool layer for expressing which resources and egress services a component may hold.	Runtime capability confinement is not hardware memory confidentiality or a non-resurrectable machine epoch.
MECI	State-dependent high-to-egress reachability prohibition plus SafeOpen and epoch revocation.	Composes with the systems above.	Targets the transition between protected computation and externally observable authority.

8. A comparison with CHERI, seL4, and WASI, and a statement of MECI’s relation to the author’s prior work on epoch isolation (MIAM) and influence-versus-authority reasoning (ACN).

Section 2 positions MECI against prior work. Sections 3–5 define the threat model, the formal model, and the epoch interlock. Section 6 proves the safety results and Section 7 the conditional noninterference theorem. Section 8 reports model checking. Sections 9–14 cover ambient channels, platform refinement, architecture, agent memory semantics, performance, and failure handling. Sections 15–17 list open obligations, limitations, and the release boundary.

2. Research Position and Relation to Prior Work

Information-flow security. Noninterference states end-to-end confidentiality as indistinguishability of low observations across runs that differ only in high inputs [4, 5]. Useful systems release some secret-dependent information, so controlled declassification is a separate mechanism with its own theory [6, 7]. The LLMbda calculus gives agent planner loops, tool calls, and LLM conversations an information-flow semantics with a noninterference theorem [1]. MECI claims no novelty in declassification theory. Its contribution is a hardware-oriented enforcement boundary that constrains *when* unreleased high state may coexist with egress authority.

Capability systems. CHERI adds unforgeable, tagged capabilities with bounds, permissions, provenance, and monotonic restriction to general-purpose ISAs [8]. seL4 combines capability-based resource control with machine-checked functional correctness and information-flow proofs for supported configurations [9, 10]. WASI grants host resources through link-time imports and unforgeable runtime handles [11]. Table 1 summarizes how each relates to MECI. These systems can supply the concrete authority graph from which Reach_s is computed; MECI adds a temporal cross-domain rule bound to cryptographic epochs and explicit release.

Hardware information flow. SecVerilog and later hardware information-flow work show that timing-sensitive noninterference can be expressed and verified at the hardware-design level [12, 13]. In a MECI

refinement, such tools can verify that RTL or microarchitectural transitions preserve the labels and observational equivalence that the abstract graph assumes.

Confidential computing and its side channels. Arm CCA with Memory Encryption Contexts, Intel TDX, AMD SEV-SNP, and NVIDIA confidential computing provide isolation, attestation, and memory or device protection in different combinations [14, 15, 16, 17]. They are enabling substrates, not substitutes for the temporal reachability invariant. CipherSteal recovers neural-network inputs from ciphertext write behavior of TEE-shielded execution [18]; TEE.fail breaks Intel and AMD confidential-computing guarantees through DDR5 memory-bus interposition [19]; TDXRay builds cache-line-granular host-side access traces of unmodified Intel TDX guests from four new side-channel primitives [20]; a recent survey catalogs memory side-channel classes more broadly [21]. These results motivate MECI’s explicit ambient-channel partition rather than any claim that egress gating closes all observation channels.

Relation to the author’s prior work. Mission-Invariant Architecture Morphing (MIAM) binds service authority to cryptographic epochs that are retired when the service graph changes [22]. MECI applies the same epoch discipline to the memory-egress boundary and adopts MIAM’s injective tuple encoding for key-derivation inputs (Section 5). Attack Calculus (ACN) separates influence over an AI agent from the authority the agent holds and checks authority with a capability order [23]. The rogue-agent walkthrough in Section 7.3 is an instance of that separation: a planner that an attacker fully influences still lacks the authority to change labels, widen the release policy, or commit the egress mask.

3. Threat Model and Security Objective

3.1 Adversary

The modeled adversary may control or manipulate agent-level software behavior, including prompts, tool descriptions, retrieved content, planner decisions, and tool-call parameters, and may induce arbitrary sequences of allowed software operations. In stronger deployment profiles, the host operating system or hypervisor may also be untrusted, subject to the guarantees of the selected confidential-computing platform.

The adversary is assumed not to break the underlying cryptographic primitives. Whether physical access, supply-chain compromise, power analysis, electromagnetic observation, or memory-bus interposition is in scope is a separate profile decision. This matters because modern TEEs intentionally exclude some physical attacks. TEE.fail demonstrates practical DDR5 interposition against Intel TDX and AMD SEV-SNP [19], and both vendors responded that such physical attacks are outside the published threat models of the affected TEEs [24, 25].

3.2 Two channel classes

Let $\mathcal{C}$ be the set of attacker-observable channels. We partition it as

$$\mathcal{C} = \mathcal{C}_g \cup \mathcal{C}_a, \quad (2)$$

where $\mathcal{C}_g$ is the set of *gated channels* and $\mathcal{C}_a$ the set of *ambient channels*. Typical members of $\mathcal{C}_g$ include network transmit paths, MCP or other tool invocations, file writes that cross the trust boundary, clipboard and IPC paths, device output queues, and explicit export APIs. A reference monitor can mediate these.

Typical members of $\mathcal{C}_a$ include cache timing, secret-dependent memory access patterns, encrypted-memory write patterns visible to a malicious host, shared-resource contention, physical bus observations, power, electromagnetic radiation, and acoustic leakage. Ambient observations made *during* protected computation cannot be repaired by purging memory afterward; CipherSteal and TDXRay demonstrate this class directly [18, 20].

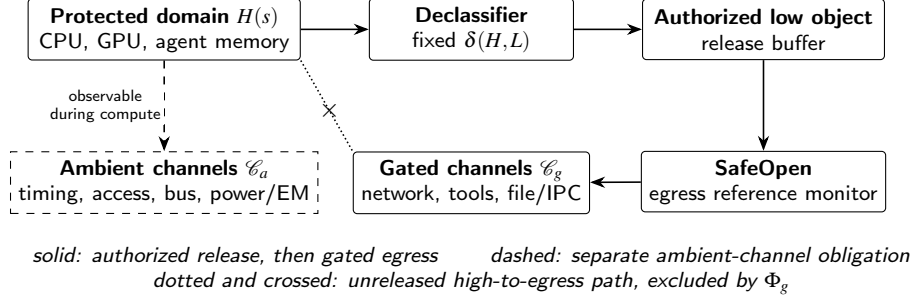


Figure 1: Capability-cut view of MECI. Protected information may reach gated egress only through an authorized declassifier and the SafeOpen reference monitor. Ambient observations are a separate security obligation because they can occur during protected computation.

4. Formal Model

4.1 Objects, influence graph, and reachability

Let S be the set of machine states. For each $s \in S$, define a directed, state-dependent influence graph

$$\mathcal{G}_s = (V_s, A_s, \ell_s), \quad (3)$$

where V_s contains information-bearing objects, active authorities, declassifier nodes, and attacker-observable channel endpoints; $A_s \subseteq V_s \times V_s$ is the direct influence relation; and $\ell_s : V_s \rightarrow \{H, L, D\}$ labels each vertex high, low, or authorized declassifier.

A direct edge $x \rightarrow_s y$ exists exactly when the active configuration permits information in x to affect y without an intervening security decision. The relation covers explicit and implicit dependencies at the model boundary: loads and stores, aliases, shared buffers, queue descriptors, DMA mappings, device command streams, tool-call argument construction, capability delegation, control decisions whose outcome is externally visible, and scheduler or timing edges when those observations are in the deployment profile. A platform refinement is sound only if every concrete information path in the deployment threat model induces a corresponding edge.

Let $A_s^\delta \subseteq A_s$ be the set of edges *leaving* a D -labeled vertex whose carried value is exactly a value permitted by the release function δ (Section 7.1). Edges into a declassifier remain in the graph, so a declassifier with any unauthorized output still connects its high inputs to that output. Define the unreleased graph $\mathcal{G}_s^{-\delta} = (V_s, A_s \setminus A_s^\delta, \ell_s)$ and, for $x, y \in V_s$,

$$\text{Reach}_s(x, y) \equiv (x, y) \in (A_s \setminus A_s^\delta)^*, \quad (4)$$

where $(\cdot)^*$ is reflexive-transitive closure. For a gated channel c with endpoint $v_c \in V_s$,

$$\text{HighReach}(c, s) \equiv \exists x \in V_s : \ell_s(x) = H \wedge \text{Reach}_s(x, v_c). \quad (5)$$

Reflexivity matters: if the endpoint itself holds high data (for example, a send buffer into which a secret was copied), HighReach must be true even when no edge enters the endpoint. A transitive-only closure misses this case (Section 8, graph checks). Let $\text{Enabled}(c, s)$ mean that the endpoint can currently produce an attacker-observable effect, and write $E(s) \equiv \exists c \in \mathcal{C}_g : \text{Enabled}(c, s)$.

Definition 1 (Gated-egress invariant).

$$\Phi_g(s) \equiv \forall c \in \mathcal{C}_g : \text{Enabled}(c, s) \Rightarrow \neg \text{HighReach}(c, s).$$

This definition makes the implementation burden explicit. A newly discovered tool, alias, DMA mapping, derived buffer, or control dependency is not handled by adding prose; it must appear as a vertex or edge in $\mathcal{G}_s$. Conversely, a value that has passed through a confined declassifier leaves on an authorized edge in A_s^δ and can become low without declaring all secret-dependent state illegal.

4.2 Sealed objects

MECI keeps protected state sealed at rest while egress is open; otherwise every egress interval would require destroying all protected memory. The graph therefore needs a rule for ciphertext. If ciphertext were always high, Φ_g could not hold whenever a storage-backed tool can reach sealed state; if it were always low, a usable key would silently reconnect it to plaintext.

Assumption 1 (Sealed-object confidentiality). Sealing uses an authenticated-encryption scheme with associated data [26] that provides ciphertext indistinguishability and integrity under the deployment’s key-use limits, with nonces that never repeat under a key and ciphertext lengths padded to policy-defined size classes.

Definition 2 (Sealed-object labeling). A vertex holding an object sealed under a domain key K_e^d is labeled L in $\mathcal{G}_s$ if no key able to open it is usable in s (Section 5) and its size class is part of the authorized release; otherwise it is labeled H .

Deterministic memory encryption inside a TEE is not sealing in this sense. Its ciphertext is observable while the key is live, so those observations belong to $\mathcal{C}_a$ (Section 9).

4.3 Strict finite projection

For model checking and implementation profiles, a finite Boolean projection is useful. Let

- D : the current-epoch decryption capability τ_e is not yet revoked,
- P : readable protected plaintext remains architecturally resident,
- M : secret-dependent microarchitectural residue remains,
- I : a DMA-capable path can reach protected memory,
- G : secret-dependent accelerator or device state remains,
- B : an unreleased high object exists in an exportable buffer,
- Q : a high-producing execution unit can still create high state.

The strict finite invariant is

$$\Phi_{bit}(s) = \neg(E \wedge (D \vee P \vee M \vee I \vee G \vee B \vee Q)). \quad (6)$$

B closes the “copy the secret, purge memory, then send the copy” counterexample (Proposition 3). Q prevents a racing core, accelerator, or DMA producer from recreating high state between cleanup and egress enable. The projection is useful only if clearing the bits really removes high reachability, which we state as an obligation on the implementation profile.

Assumption 2 (Projection soundness). For every reachable s : if $\neg(D \vee P \vee M \vee I \vee G \vee B \vee Q)$ holds in s , then $\forall c \in \mathcal{C}_g : \neg \text{HighReach}(c, s)$.

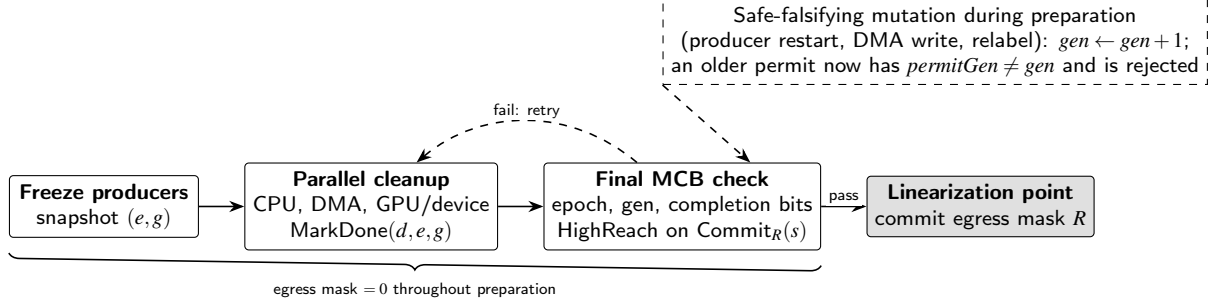


Figure 2: Concrete *SafeOpen* transaction. Cleanup may be multi-step and concurrent, but the final check and the egress-mask commit must be one linearized step of the trusted monitor. A generation counter invalidates stale safety snapshots.

4.4 Safety predicate and linearizable *SafeOpen*

Define

$$\text{Safe}(s) \equiv \neg D \wedge \neg P \wedge \neg M \wedge \neg I \wedge \neg G \wedge \neg B \wedge \neg Q \wedge \text{FreshEpoch}(s). \quad (7)$$

The only operation that may change a gated channel from disabled to enabled is *SafeOpen*. For a requested channel set $R \subseteq \mathcal{C}_g$, let $\text{Commit}_R(s)$ be the configuration that equals s except that the egress mask is R . The final check is evaluated on that post-commit configuration:

$$\text{SafeOpen}(s, s', R) \Rightarrow E(s) = 0 \wedge \text{Safe}(s) \wedge s' = \text{Commit}_R(s) \quad (8)$$

$$\wedge \forall c \in R : \neg \text{HighReach}(c, s') \wedge E_R(s') = 1. \quad (9)$$

Enabling a channel can itself activate edges, for example by mapping a buffer into a NIC descriptor ring. Because Commit_R changes only the egress mask, the monitor can compute $\mathcal{G}_{s'}$ from $\mathcal{G}_s$ and R before committing, so evaluating the check on s' is implementable.

MECI Control Block. A concrete design uses a small trusted MECI Control Block (MCB) below the hostile OS and agent boundary. The MCB owns the hardware-visible egress mask and stores at least

$$\langle \text{epoch}, \text{gen}, \text{complete}, \text{egressMask}, \text{policyHash}, \text{releaseDigest} \rangle. \quad (10)$$

Every trusted-state mutation that can falsify *Safe* or make *HighReach* true increments *gen*. Domain cleanup sets bits in *complete* only for the current generation. The MCB exposes four primitives:

1. *Freeze*(e): disable new high-producing work and return the current generation g .
2. *MarkDone*(d, e, g): record completion for a required domain d only if e and g are still current.
3. *SafeOpen*(R, e, g, r): recompute the final safety predicate and *HighReach* on $\text{Commit}_R(s)$, require the expected epoch and generation, verify the required completion bits and the release digest r , then commit the egress mask.
4. *Close*($\emptyset$): clear the egress mask before any protected capability can be issued for a new epoch.

The linearization point is the trusted commit that changes *egressMask* from zero to R after the final validation. A check performed earlier in software is not the linearization point. If any high-producing state changes after a preparatory check, *gen* changes and the stale permit cannot commit. Failure, timeout, reset, or crash before the commit leaves *egressMask* = 0.

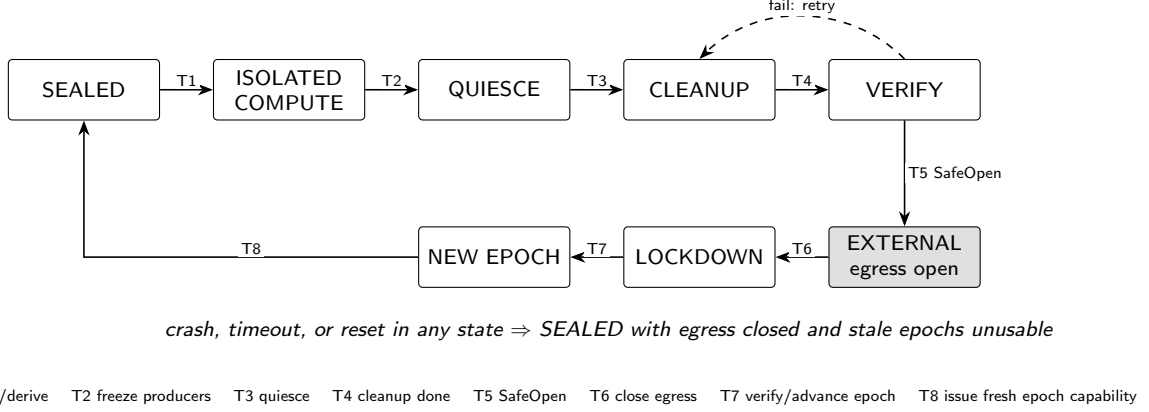


Figure 3: MECI lifecycle. *EXTERNAL (shaded) is the only state with egress open. Cleanup and verification may retry while egress stays closed; failures return to SEALED. The security-critical linearization point is T5.*

Proposition 1 (Stale-snapshot failure). *If a system evaluates $\text{Safe}(s)$, records an unversioned permit, later allows a protected producer to restart, and then enables egress using only that permit, the gated-egress invariant can be violated even though the earlier check was correct.*

Proof. Reach a state with all hazards cleared and evaluate $\text{Safe} = \text{true}$. Before egress commits, restart a producer so $Q := 1$. If this mutation does not invalidate the permit, the split open step sets $E := 1$ while $Q = 1$, so Φ_{bit} is false. A generation-checked permit prevents the final step because $\text{permitGen} \neq \text{gen}$ after the restart. Both TLC and the reference checker return this trace as the shortest counterexample for the unversioned design (Section 8). $\square$

4.5 Lifecycle

Figure 3 shows the lifecycle. Egress is open in exactly one state, EXTERNAL. Every other state, including failure paths, keeps the egress mask at zero. A useful forward partial order is

$$\text{Freeze} \prec \text{Quiesce} \prec \text{VerifySafe} \prec E\uparrow, \quad (11)$$

with revocation, DMA quarantine, memory cleanup, and device cleanup complete before VerifySafe succeeds. Their internal order can vary if no step can reopen egress and no producer can recreate high state.

5. Cryptographic Epoch Interlock

Let K_R be a root secret held by a hardware root of trust or key-management service, $e \in \mathbb{N}$ the isolation epoch, c a monotonic anti-rollback counter, τ_e a non-exportable epoch capability, σ_e a per-epoch seed held inside the key service, d a domain separator, π_e a canonical policy description, and A_e an attested configuration measurement. Domain keys are derived as

$$K_e^d = \text{KDF}(K_R, \langle \text{"MECI-v1"}, e, c, \sigma_e, d, H(\pi_e), H(A_e) \rangle). \quad (12)$$

Here $\langle \cdot \rangle$ is the injective tuple encoding used in MIAM [22]: each field carries a fixed-width type tag and a 32-bit big-endian length followed by its bytes, and the number of fields is fixed per context label. Raw concatenation is never used, so no two distinct input tuples share an encoding. The capability τ_e is not itself a KDF input; it gates whether the key service will evaluate Equation (12) for epoch e at all. The policy

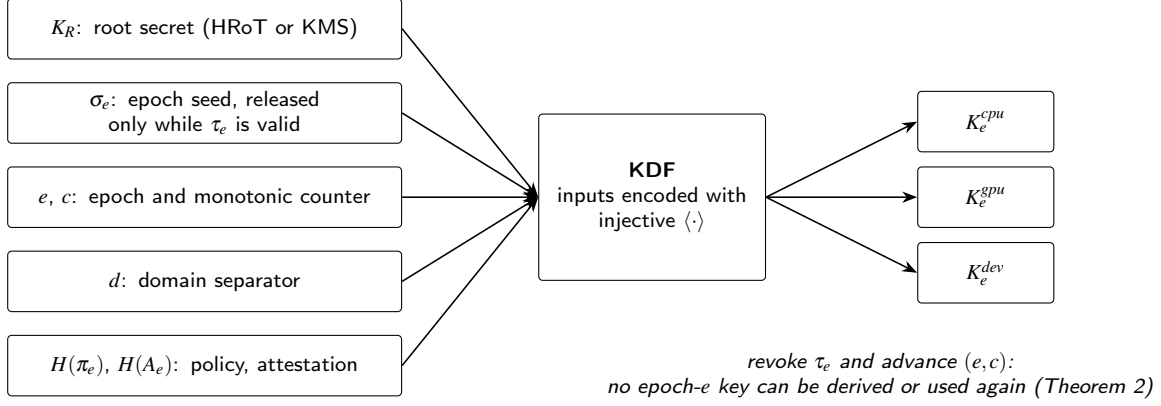


Figure 4: Epoch and domain key separation. All inputs enter the KDF in parallel through an injective encoding. The construction is abstract; no current TEE is claimed to expose these exact key controls.

and attestation bindings prevent a valid epoch from being replayed under a different egress policy, software measurement, or device configuration.

Key use is governed by the rule

$$\text{Usable}(K_e^d, s) \Rightarrow \text{Valid}(\tau_e, s) \wedge e = \text{CurrentEpoch}(s) \wedge c = \text{Counter}(s), \quad (13)$$

and the projection bit D holds exactly when $\neg \text{Revoked}(\tau_{\text{CurrentEpoch}(s)}, s)$. This is how the TLA+ model in Section 8 derives D .

Assumption 3 (Irreversible revocation). $\text{Revoked}(\tau_e) \Rightarrow \Box \neg \text{Valid}(\tau_e)$.

Assumption 4 (Monotonic epoch). The persistent epoch and counter state, held in rollback-protected storage, never decreases: $e_{t+1} \geq e_t$ and $c_{t+1} \geq c_t$.

6. Safety Results

The safety theorem rests on two local obligations. Neither is a lemma about source code; each is a property the trusted implementation must establish.

Obligation 1 (Local egress guard). For every transition $s \rightarrow s'$ and channel c with $\neg \text{Enabled}(c, s) \wedge \text{Enabled}(c, s')$: $\text{Safe}(s)$ holds, $\neg \text{HighReach}(c, s')$ holds, and the check and the commit form one linearized step of the trusted monitor, so no transition that can falsify Safe is linearized between them.

A software check followed by an independently writable NIC register does not satisfy Obligation 1 under a hostile kernel. The check and enable must be mediated by the trusted reference monitor, firmware, hypervisor extension, device security controller, or an equivalent trusted component.

Obligation 2 (Reachability creation). For every transition $s \rightarrow s'$ and channel $c \in \mathcal{C}_g$: $\neg \text{HighReach}(c, s) \wedge \text{HighReach}(c, s') \Rightarrow \neg \text{Enabled}(c, s')$.

Obligation 2 covers every way HighReach can become true: a new edge, a new alias or mapping, a write of high data into an already-connected buffer (which relabels it), or a key becoming usable for a sealed object (Definition 2).

Theorem 1 (Gated-egress safety). Let $\Sigma = (S, \rightarrow, s_0)$ be a transition system with $\Phi_g(s_0)$. If every transition satisfies Obligations 1 and 2, then every reachable state satisfies Φ_g .

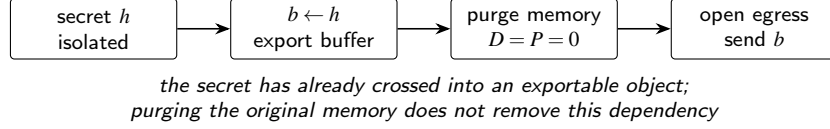


Figure 5: Memory-egress mutual exclusion can hold while a pre-staged secret leaks. The fix is the B hazard, which classifies unreleased export objects as high and requires explicit declassification.

Proof. Induction on execution length. The base case is $\Phi_g(s_0)$. Assume $\Phi_g(s)$ and $s \rightarrow s'$, and let $c \in \mathcal{C}_g$ with $\text{Enabled}(c, s')$. If $\neg \text{Enabled}(c, s)$, the transition enables c , and Obligation 1 gives $\neg \text{HighReach}(c, s')$. If $\text{Enabled}(c, s)$, the induction hypothesis gives $\neg \text{HighReach}(c, s)$; were $\text{HighReach}(c, s')$ true, Obligation 2 would give $\neg \text{Enabled}(c, s')$, a contradiction. In both cases $\neg \text{HighReach}(c, s')$, so $\Phi_g(s')$. $\square$

Proposition 2 (Projection refinement). *Under Assumption 2, $\Phi_{bit}(s) \Rightarrow \Phi_g(s)$ for every reachable s .*

Proof. Let $\text{Enabled}(c, s)$. Then $E(s) = 1$ by the definition of E , so $\Phi_{bit}(s)$ forces $D = P = M = I = G = B = Q = 0$. Assumption 2 then gives $\neg \text{HighReach}(c', s)$ for every $c' \in \mathcal{C}_g$, in particular for c . $\square$

Proposition 2 is what connects the finite model checked in Section 8 to the graph invariant: the checker establishes Φ_{bit} within its bounds, and projection soundness transfers the result to Φ_g .

Theorem 2 (Epoch non-resurrection). *Assume the key-use rule (13) and Assumptions 3 and 4. If τ_e is revoked in state s_r , then no K_e^d is usable in any state reachable from s_r .*

Proof. Suppose $\text{Usable}(K_e^d, s')$ for some s' reachable from s_r . By (13), $\text{Valid}(\tau_e, s')$. Assumption 3 gives $\neg \text{Valid}(\tau_e)$ in every state from s_r onward, a contradiction. Assumption 4 separately prevents a rollback from restoring e and c as the current epoch and counter, which (13) also requires. $\square$

Corollary 1 (Egress only after permanent revocation). *Every SafeOpen for epoch e occurs after τ_e is revoked, so no epoch- e domain key is usable during or after any egress interval of that epoch.*

Proof. Safe requires $\neg D$, and D holds exactly while τ_e for the current epoch is unrevoked. Theorem 2 makes the revocation permanent. $\square$

7. Why Safety Is Not Noninterference

The original formulation inferred noninterference from the fact that protected memory was no longer readable when egress opened. That inference is invalid. Noninterference is relational: two executions that differ only in protected input must be indistinguishable at low observations [4, 5], except for explicitly authorized release [6, 7].

Proposition 3 (Export-buffer counterexample). *There is a system satisfying memory-egress mutual exclusion (1) whose egress output reveals the secret.*

Proof. Let $h \in \{0, 1\}$ be protected. During isolated computation, execute $b := h$ into an exportable buffer. Revoke the protected-memory key and purge protected memory, so $D = P = 0$. Enable egress and transmit b . The invariant $E \Rightarrow D = P = 0$ holds throughout, yet the external observation equals h . $\square$

7.1 Authorized release and a concrete agent policy

Let H denote protected input and L public input. An authorized release function

$$\delta : \mathcal{H} \times \mathcal{L} \rightarrow \mathcal{R} \quad (14)$$

is part of the security policy, not a model choice. Inputs are release-equivalent, written $(H_1, L) \approx_\delta (H_2, L)$, when $\delta(H_1, L) = \delta(H_2, L)$.

For a representative incident-triage agent, let H contain raw customer logs, conversation text, credentials and tokens, internal analyst notes, and detailed infrastructure identifiers. A mechanically enforceable release is a fixed schema such as

$$\delta(H, L) = \langle caseId(L), severityBucket(H), tacticClass(H), approvedIndicators(H) \rangle. \quad (15)$$

The policy permits categorical severity, a bounded tactic classification, and a separately approved set of indicators, and forbids raw credentials, unrestricted log text, and hidden memory. A free-form LLM summary is *not* automatically low. It remains high until a deterministic policy verifier or an explicitly authorized reviewer accepts it as a release object, which prevents the model from silently becoming the declassifier.

7.2 Release equivalence and conditional noninterference

Let $\text{Low}(s)$ denote the complete low architectural state together with the set of released objects visible to gated channels. Define

$$s_1 \mathcal{R}_\delta s_2 \iff \text{Low}(s_1) = \text{Low}(s_2) \wedge \delta(H_1, L) = \delta(H_2, L). \quad (16)$$

Let $\text{Obs}_\mathcal{C}(\pi)$ be the full modeled observation trace of run π . Abstract transitions are partitioned into high-internal steps τ_H , authorized release steps τ_δ , gated-output steps τ_g , and ambient-observation steps τ_a .

The theorem needs one semantic property of the graph: that it really captures dependence.

Assumption 5 (Influence-graph soundness). For every state s and gated channel c , the value emitted on c by a gated-output step from s is a function of the values held at the vertices that reach v_c in $\mathcal{G}_s^{-\delta}$ and of the values carried into those vertices along authorized edges in A_s^δ .

Lemma 1 (Low determinism of gated output). *If $\Phi_g(s)$ and Assumption 5 hold, every value emitted by a gated-output step from s is a deterministic function of $\text{Low}(s)$.*

Proof. Let c be enabled in s and let U be the set of vertices that reach v_c in $\mathcal{G}_s^{-\delta}$. By $\Phi_g(s)$, U contains no H -labeled vertex. U is closed under predecessors in $\mathcal{G}_s^{-\delta}$, so no H -labeled vertex has a path into U in that graph either; in particular every declassifier in U has no high input. Every value held in U is therefore determined by L -labeled values and by values entering U along authorized edges, which are release objects. Both are part of $\text{Low}(s)$, and Assumption 5 makes the emitted value a function of them. $\square$

Theorem 3 (Noninterference modulo authorized release). *Consider two runs with the same public input L and $(H_1, L) \approx_\delta (H_2, L)$. Suppose:*

1. *the initial states satisfy $\mathcal{R}_\delta$;*
2. *every τ_H step leaves low observations unchanged and can be matched by zero or more τ_H steps in the other run while preserving $\mathcal{R}_\delta$;*
3. *every τ_δ step updates low state only as a deterministic function of the prior low state and the release value $\delta(H, L)$;*

4. every reachable state satisfies Φ_g (for example, by Theorem 1) and Assumption 5 holds;
5. for every in-scope ambient channel a with observation function λ_a , $s_1 \mathcal{R}_\delta s_2 \Rightarrow \lambda_a(s_1) = \lambda_a(s_2)$;
6. scheduling, termination, timing, and nondeterministic choices are either included in condition 5 or constrained to be low-deterministic; and
7. the concrete implementation refines the abstract transition and reachability model.

Then $\text{Obs}_\mathcal{C}(\pi_1) = \text{Obs}_\mathcal{C}(\pi_2)$.

Proof. Induct on the number of attacker-observable events. Initially the states are related by condition 1. A high-internal step changes no low observation and, by condition 2, can be stutter-matched while preserving $\mathcal{R}_\delta$. For an authorized release step, both runs release the same value by release equivalence, so condition 3 yields equal next low states and preserves the relation. For a gated-output event, condition 4 and Lemma 1 make the emitted value a function of $\text{Low}(s)$, which is equal in related states, so the outputs match. For an ambient event, equality follows from condition 5. Condition 6 preserves the correspondence for scheduling, timing, termination, and other modeled nondeterminism. After each observable event the pair remains $\mathcal{R}_\delta$ -related and the observation prefixes are equal, so the complete modeled traces are equal. Condition 7 transfers the abstract result to an implementation only once a refinement proof or equivalent evidence exists. $\square$

In this form the MECI invariant does real work: the gated-output step follows from Φ_g through Lemma 1 instead of being assumed. The remaining engineering burden sits in conditions 2, 5, 6, and 7 and in Assumption 5, which is the semantic content of reachability soundness (obligation P2 in Table 4). MECI does not turn an unmodeled physical or microarchitectural leakage channel into a gated one.

7.3 Rogue-agent walkthrough

Suppose an injected instruction causes an agent to read a protected API token and call an MCP tool with the token embedded in a URL. During isolated computation, the token object and every derived tool-argument object are high. The tool endpoint is a gated channel. When the call is requested, the MCB freezes new high-producing work, quiesces asynchronous producers, and evaluates the post-commit graph. Because the raw token still reaches the tool-argument vertex, $\text{HighReach}(\text{tool}, \text{Commit}_R(s)) = 1$ and `SafeOpen` rejects the request. If policy permits only the structured release (15), the declassifier constructs that low object, and the tool may send it once the graph contains no unreleased high-to-tool path. A rogue planner can *request* leakage, but, as in ACN’s separation of influence from authority [23], it holds no authority to change labels, widen δ , or commit the egress mask.

8. Model Checking: TLA+ and an Independent Reference Checker

The finite safety core is specified in TLA+ [27, 28] as `spec/MECI.tla` and implemented a second time, independently, as a dependency-free Python checker, `scripts/meci_modelcheck.py`. Both use the same state variables (egress flag, hazard set over P, M, I, G, B, Q , generation counter, permit and permit generation, current epoch, revoked-epoch set), the same initial state (isolated computation with every hazard present), and the same actions: per-hazard cleanup, revocation of the current epoch (which clears the derived D), production of new high state by a live producer, producer restart, the design-specific open, and close-and-advance into a new isolated epoch. Every action that can falsify `Safe` increments the generation. The *FreshEpoch* conjunct of (7) is represented through the derived D and checked separately by `EpochInvariant` and the action property `NoResurrection`.

Table 2: Exhaustive results within $GenBound = 3$, $MaxEpoch = 3$. TLC 2.19 and the independent reference checker report identical values in every numeric column. Invariant checked: $egress \Rightarrow Safe$.

Design	Distinct	Generated	Depth	Violating	Shortest counterexample
Atomic SafeOpen	705	3,085	21	0	none
Generation-checked split	858	3,442	24	0	none
Unversioned split (buggy)	938	3,700	24	70	10 steps: clean six hazards, revoke, check, restart producer, open on stale permit
D/P -only guard	980	4,418	15	275	3 steps: clean P , revoke, open with B, G, I, M, Q present
Guard omitting B	712	3,103	19	7	7 steps: clean G, I, M, P, Q , revoke, open with staged export object

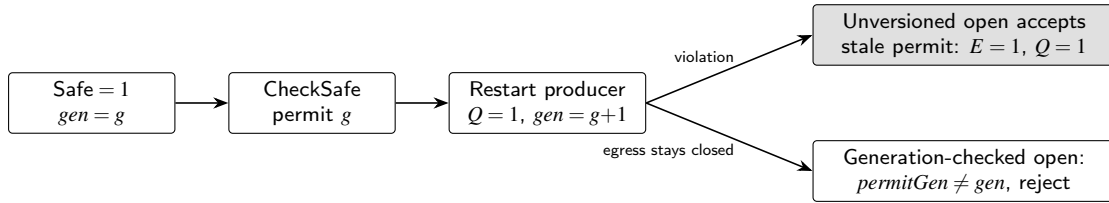


Figure 6: Shortest stale-snapshot class, found by both TLC and the reference checker. The decision happens at the open step: the unversioned design accepts the stale permit, while generation validation rejects it after protected work restarts.

Five open designs are checked. *Atomic* checks Safe and commits in one step. *Gencheck* splits check and commit but validates the generation. *Buggy* splits them without validation. *DP-only* checks only D and P , as in Equation (1). *No-B* checks every hazard except B . Bounds are $GenBound = 3$ and $MaxEpoch = 3$; within them the search is exhaustive.

Table 2 reports the results. The two implementations agree exactly on distinct states, generated states, search depth, and the number of violating states for all five designs, which guards against a modeling error in either one. TypeOK, PermitInvariant ($FreshPermit \Rightarrow Safe$), EpochInvariant, and NoResurrection hold in every design. Raising the bounds in the reference checker to (4, 3) and (5, 4) grows the state space to as many as 3,392 distinct states and leaves the outcome unchanged: zero violations for the two correct designs and the same shortest counterexamples for the three weakened ones.

The checker also runs three non-temporal controls. *Release equivalence*: sixteen protected records ($severity \in \{0..3\}$, $tactic \in \{0, 1\}$, $token \in \{0, 1\}$) with $\delta = (severity \div 2, tactic)$. An export of exactly δ produces zero release-equivalence violations; an export that also carries the token, and one that carries raw severity, each produce 32 ordered violating pairs. *Epoch rollback*: with revocation and counter in rollback-protected storage, none of eight revoked epochs becomes usable after a reboot from a pre-revocation snapshot; when both are restored from that snapshot, all eight are resurrected. *Graph checks*: a transitive-only closure misses a high-labeled endpoint that the reflexive closure in (4) detects; relabeling a connected low buffer as high makes HighReach true, which is exactly the event Obligation 2 must block; and removing the authorized declassifier edge removes reachability.

This is bounded model checking of the abstract reference model. It is not a proof about a processor, firmware stack, GPU, NIC, or TEE, and the relational and rollback controls are small illustrative tests rather than proofs.

9. Ambient Channels and the Side-Channel Boundary

9.1 Ciphertext and microarchitectural observations

CipherSteal shows that a malicious host can recover neural-network inputs from the ciphertext write behavior of TEE-shielded execution [18], and TDXXRay reconstructs cache-line-granular access traces of unmodified TDX guests from host-observable signals [20]. Such observations occur while protected computation is active, so a variable such as “ciphertext residue present” cannot be made safe by clearing it before egress. The obligation must be stated as an ambient-channel property during the protected epoch:

$$s_1 \mathcal{R}_\delta s_2 \Rightarrow \lambda_a(s_1) = \lambda_a(s_2) \quad (17)$$

for each modeled ambient channel a . Candidate mitigations include access-pattern-oblivious algorithms, partitioned caches, hardware information-flow enforcement, secret-aware compilation, ORAM-style access hiding, probabilistic re-randomization for ciphertext channels, or excluding the observation from the deployment threat model with an explicit physical assumption. The right choice depends on the platform and the attacker.

9.2 Physical memory-bus interposition

TEE.fail shows that current confidential-VM protections do not make the physical memory bus unobservable, exploiting the deterministic memory encryption these platforms use [19]; broader surveys show leakage across cache, memory-controller, interconnect, and physical observation classes [21]. The lesson is not “replace one cipher mode and the problem disappears”: memory-encryption systems have architecture-specific address binding, replay, integrity, key, and tweak requirements.

The baseline MECI theorem therefore excludes physical bus interposition unless a deployment explicitly promotes the bus to an in-scope ambient channel and supplies a separate low-equivalence argument. A higher-assurance “MECI-Physical” profile could require controlled physical custody, tamper evidence or detection, protected links or memory technology designed for the selected physical attacker, and a platform-specific proof that bus observations are release-equivalent. This is a deployment profile, not a consequence of the gated-egress interlock.

10. Hardware Refinement Targets

Current confidential-computing platforms provide useful primitives, but none is described here as already implementing the MECI state machine. The research task is a refinement proof from platform behavior to the abstract model (Table 3).

10.1 Arm CCA/RMM refinement proposal

Arm describes MEC as a Realm Management Extension feature in which memory accesses are associated with distinct encryption contexts through MECIDs, and RME device assignment extends isolation to assigned devices through the SMMU [14, 29]. A MECID is a useful context primitive but is not identified with τ_e here.

A first implementation target is an RMM or secure-firmware extension that maintains per-Realm MECI state. A proposed RMI_MECI_SAFE_OPEN call would: (1) require all Realm vCPUs at a defined quiescent point; (2) require SMMU and device-assignment state to deny protected-memory access from egress-capable devices; (3) verify the current epoch and MCB generation; (4) validate the completion bitmap and release

Table 3: *Candidate platform primitives and remaining MECI obligations.*

Platform	Supported primitive	MECI-specific obligation
Arm CCA + MEC	Multiple Realm memory-encryption contexts identified by MECIDs; Realm attestation; RMM-mediated lifecycle; device assignment through the SMMU [14, 29].	Map epoch capabilities to MECID allocation and revocation, gate egress devices, make stale contexts unreachable, and state which microarchitectural state is covered.
Intel TDX	Hardware-isolated Trust Domains, TME-MK memory encryption, Secure EPT and PAMT, remote attestation; Intel documents cache side-channel considerations for TDX [15, 30].	Add a trusted egress reference monitor and linearizable SafeOpen; show that I/O, shared memory, cache, migration, and device behavior refine HighReach.
AMD SEV-SNP	Guest memory confidentiality and integrity, RMP, attestation, PSP-managed firmware [16].	Include IOMMU and MMIO paths in the refinement, account for vendor bulletins such as the February 2026 SEV-SNP IOMMU findings [31], and supply rollback-protected epoch state that SNP does not provide by itself.
NVIDIA H100 CC	GPU attestation, on-die root of trust, protected execution mode, firewalls, encrypted CPU-GPU transfers through bounce buffers. On-package HBM is treated as physically protected and is not encrypted [32, 17].	Treat resident GPU state as protected by access control, not by an HBM key. Define quiescence, context destruction and reset, device DMA boundaries, and the physical-HBM threat profile.

digest; (5) recompute or conservatively validate the post-commit high-to-egress graph; and (6) commit a secure egress-capability bitmap at the final linearization point. `RMI_MECI_CLOSE` would clear egress authority before a fresh epoch capability is issued. These calls are a proposed interface, not current Arm architecture.

The refinement proof must show that the concrete RMM, SMMU, device-assignment, memory-context, and firmware states jointly implement Safe, HighReach, epoch freshness, and failure closure. This is where a formal seL4- or CHERI-style authority graph, or a UCLID5 hardware model [13], would reduce ambiguity.

10.2 Intel and AMD: memory confidentiality is not an egress interlock

Intel TDX isolates a Trust Domain from the VMM and encrypts private memory with TME-MK [15], and Intel publishes guidance on cache-based side channels that affect TDX [30]. AMD SEV-SNP provides encrypted guest memory, integrity metadata, and attestation [16], and its security bulletins show that IOMMU and firmware paths remain security-relevant [31]. Neither platform states the temporal property “egress enabled implies no unreleased high state is reachable.” That is the additional MECI control objective.

10.3 NVIDIA Hopper and the GPU quiescence contract

An earlier draft of this work modeled H100 confidential computing as encrypting HBM under a rotatable key. NVIDIA’s description is different: in confidential-computing mode the GPU computes on plaintext in on-package HBM, which is treated as physically protected and is not encrypted, while transfers across the CPU-GPU interconnect are encrypted and authenticated [32, 17]. A Hopper refinement therefore cannot discharge G by destroying an HBM key.

For MECI, GPU quiescence means more than “the host called synchronize.” The trusted monitor must establish a platform-specific predicate: no protected kernel is executing; no protected command can still write an egress-reachable buffer; all copy and DMA engines are idle or confined to low buffers; no new queue submission can occur for the protected context; device page tables and SMMU permissions deny

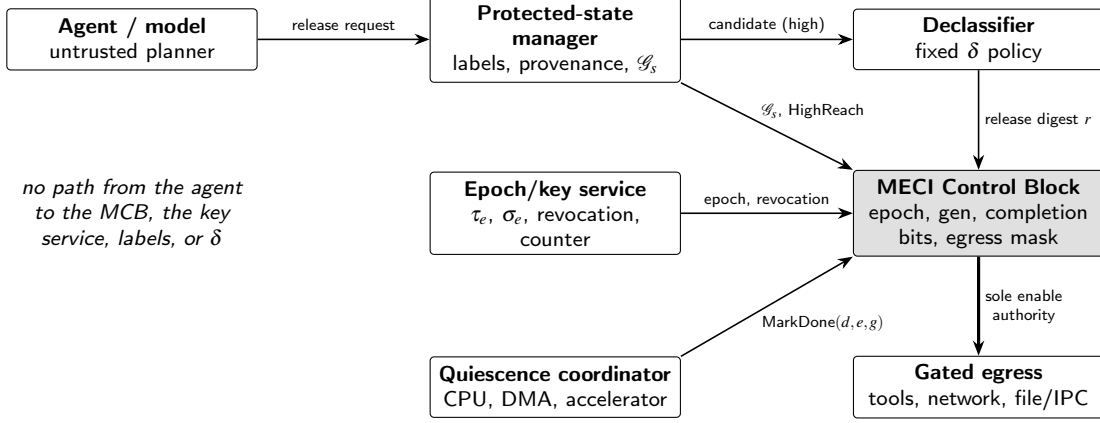


Figure 7: Reference architecture. The agent can request a release but cannot change labels, the declassification policy, generation or epoch state, or the egress mask. Only the MECI Control Block can enable gated egress.

high-to-egress DMA; and context and reset semantics prevent stale registers or allocations from influencing a subsequently enabled egress domain. If the platform cannot establish these facts against the selected adversary, GPU state stays in $H(s)$ and SafeOpen must fail.

11. Reference Architecture

A practical MECI implementation decomposes into five trusted mechanisms (Figure 7).

1. **Protected-state manager.** Maintains conservative labels, provenance, and the state-dependent influence graph across CPU memory, accelerators, device buffers, agent memory stores, and export objects.
2. **Quiescence coordinator.** Freezes high-producing cores, threads, accelerators, DMA engines, callbacks, and asynchronous tool workers before the final egress check, and reports completion per domain.
3. **Epoch/key service.** Issues τ_e , holds σ_e , binds domain keys to epoch, policy, and attestation, revokes old capabilities, and maintains rollback-protected epoch and counter state.
4. **Declassifier/export builder.** Constructs low release objects only through a confined δ policy and keeps candidate summaries or tool arguments high until validation completes.
5. **MECI Control Block (egress reference monitor).** Owns the egress-enable authority, generation counter, completion bitmap, and linearizable SafeOpen commit.

12. AI Agent Memory Semantics

MECI classifies memory by information dependency, not by storage technology. A vector-store record, embedding, retrieved passage, KV-cache entry, compressed summary, scratchpad, GPU tensor, tool argument, or audit object is high whenever it contains or is derived from unreleased protected information, and becomes low only through a permitted release operation or, for sealed objects, through Definition 2.

For each object x , an implementation profile should track a stable identifier, label, epoch, provenance parents, current storage and compute domains, applicable release policy, and graph edges to authorities and gated channels. The operational lifecycle is

$$\text{ingest} \rightarrow \text{classify} \rightarrow \text{seal} \rightarrow \text{retrieve} \rightarrow \text{compute} \rightarrow \text{candidate} \rightarrow \delta\text{-verify} \rightarrow \text{export or re-seal.} \quad (18)$$

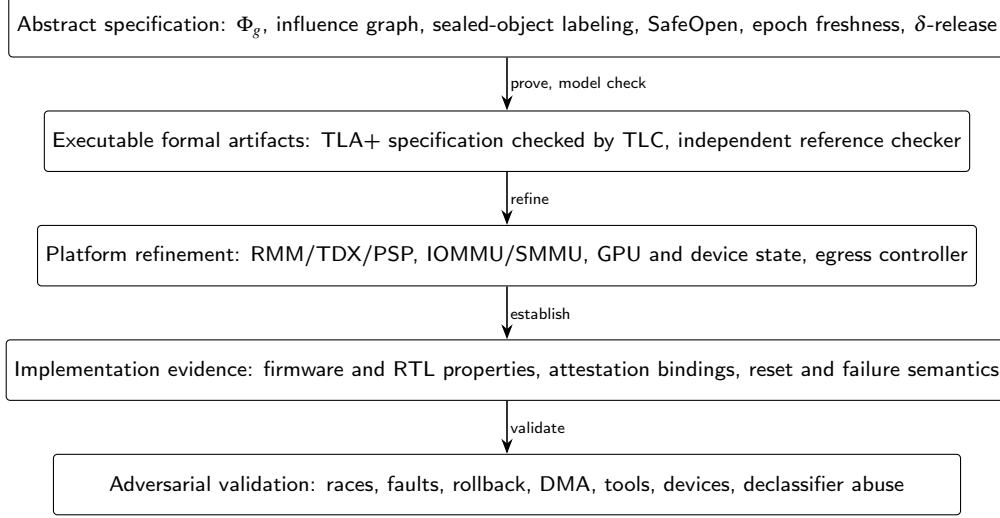


Figure 8: *Proof and validation stack. The released artifacts cover the top two layers; hardware refinement remains platform work.*

A generated summary is not low merely because it is shorter or “sanitized” by an LLM. It is a high-derived candidate until the release policy proves or approves the exact release.

Semantic egress. Tool invocation is an egress capability even if the model process never opens a socket. MCP studies show that tool metadata, high-privilege tool composition, and externally supplied tool results can steer agent behavior in ways that defeat naive least privilege [2, 3]. The reference monitor therefore mediates capabilities rather than API names. A browser, cloud SDK, shell, database connector, email action, file synchronizer, or plugin can each be a low-observation channel.

13. Performance Model

MECI’s security claim does not depend on a latency target. A defensible model decomposes a transition as

$$T_{\text{switch}} = T_{\text{freeze}} + \max(T_{\text{cpu}}, T_{\text{dma}}, T_{\text{gpu}}, T_{\text{revoke}}) + T_{\text{verify}} + T_{\text{commit}} \quad (19)$$

when independent cleanup domains run in parallel after high producers are frozen; serial implementations use the sum. The dominant term is platform-dependent and must be measured on the refinement target.

Release size matters on device paths. If the protected CPU-GPU staging path sustains bandwidth B_w , staging S bytes costs at least S/B_w before fixed latency. For an illustrative B_w of 4 GB/s, staging 1 MiB costs about 0.26 ms. This figure is a parameter for reasoning, not a measurement; encrypted bounce-buffer throughput depends on CPU, driver, and transfer size and must be measured.

Four optimizations preserve the abstract theorem: *batch release*, which accumulates several δ -approved objects per SafeOpen cycle; *parallel cleanup*, which overlaps CPU, IOMMU, device, and revocation work once $Q = 0$ and egress is closed, with completion bits checked at commit; *scoped MECI*, which enforces $\neg\text{HighReach}(c, s)$ per channel or egress domain instead of disabling unrelated low-only services; and *small release surfaces*, which prefer fixed-schema objects over large free-form exports. A reference implementation should report distributions for freeze time, device quiescence, IOMMU/SMMU invalidation, cleanup, attestation and freshness checks, declassification, and commit under idle and adversarial load. Until those measurements exist, this paper reports a cost model, not an end-to-end number.

Table 4: *Proof obligations for a publication-grade MECI implementation.*

ID	Obligation	Evidence required
P1	Gated-channel completeness	Enumerated capability inventory and proof that no unmediated low output path exists in the profile.
P2	Reachability soundness	Concrete mapping from memory, device, and tool state to Reach_s covering aliases, shared buffers, DMA, and delegation; discharges Assumptions 2 and 5.
P3	SafeOpen linearizability	Trusted ownership of the post-commit check and the egress-enable commit under concurrency, faults, and hostile software (Obligation 1).
P4	Quiescence	No CPU, GPU, device, callback, DMA engine, or asynchronous worker can recreate high state after the safety snapshot.
P5	Epoch freshness	Rollback-protected epoch and counter, irreversible revocation, stale-key rejection across restart and migration.
P6	Declassifier confinement	Only $\delta(H, L)$ becomes low, and attacker-controlled inputs cannot widen release.
P7	Ambient-channel profile	For each in-scope ambient observation, a low-equivalence proof or a separately justified mitigation.
P8	Attestation freshness	Policy and key issuance bound to measurements, nonce or freshness, device identity, and runtime state.
P9	Hardware refinement	Proof or validation that vendor primitives implement the abstract predicates the theorems use.
P10	Failure closure	Reset, crash, timeout, partial device failure, and power-cycle paths end with egress closed and stale epochs unusable.
P11	Sealed-object labeling	Sealing meets Assumption 1; size classes are part of δ ; key usability is tracked per sealed object (Definition 2, Obligation 2).

14. Failure Semantics

The system fails closed. If cleanup, attestation, or the safety check fails, the channel stays disabled and preparation may retry (Figure 3). A crash during cleanup needs no large transaction rollback, because egress authority was never granted. On restart the system enters SEALED, refuses stale epoch capabilities, reconstructs trusted state from attested measurements, and only then begins a new isolated epoch. A minimal transition audit record is

$$\langle e, c, mode, policyHash, attestationDigest, decision, timestamp \rangle, \quad (20)$$

signed or otherwise protected against undetected modification. Auditability is not part of the confidentiality proof, but it is how a violated assumption or failed refinement gets diagnosed.

15. Open Proof Obligations

Table 4 lists what an implementation must establish before it inherits the abstract results.

16. Limitations

- The model checking is bounded ($GenBound = 3$, $MaxEpoch = 3$, with sensitivity runs to $(5, 4)$) and covers the finite projection, not the full influence graph. Transfer to Φ_g rests on Assumption 2.
- Theorems 1–3 are paper proofs. No mechanized relational proof of Theorem 3 exists yet.

- There is no hardware prototype and no performance measurement; Section 13 is a cost model.
- Ambient channels and physical attacks are outside the baseline theorem unless a deployment profile proves equivalence for them.
- Declassifier correctness (P6) is assumed. LLM-generated text stays high until a verifier or authorized reviewer releases it, which limits utility for free-form outputs.
- The influence graph is only as good as its edges. An omitted concrete dependency makes the abstraction unsound, which is why P1 and P2 are first in Table 4.

17. Open Research, Reproducibility, and Release Boundary

Artifacts. The release contains the paper source (`paper/meci.tex`), the TLA+ specification and per-design TLC configurations (`spec/MECI.tla`, `spec/MECI_*.cfg`), captured TLC output (`spec/tlc_results.txt`), the reference checker (`scripts/meci_modelcheck.py`), and its captured output (`scripts/meci_modelcheck_results.txt`). SHA-256 digests are listed in Appendix A. To reproduce, run `python3 scripts/meci_modelcheck.py` and, for each design, `java -cp tla2tools.jar tlc2.TLC -deadlock -config spec/MECI_<design>.cfg spec/MECI.tla`, adding `-continue` for the three weakened designs to count every violating state.

Changes from the 10 August 2026 version. Reachability now uses reflexive-transitive closure, and sealed objects have an explicit labeling rule. SafeOpen is evaluated on the post-commit configuration. The former Lemma 1 is restated as Obligation 1, Obligation 2 is new and covers relabeling, and the proof of Theorem 1 is rewritten. Projection soundness and Proposition 2 replace a corollary that did not follow from the safety theorem. Key derivation uses a per-epoch seed and an injective encoding, with τ_e gating evaluation. Theorem 3 now derives the gated-output step from Φ_g through Lemma 1, and its ambient condition is stated on $\mathcal{R}_\delta$. The finite model is now specified in TLA+ and checked by TLC and by a rewritten reference checker; Table 2 supersedes the counts reported in the earlier version, which came from a checker with a different state encoding. Citations were corrected (vendor bulletins, the NVIDIA HBM source, survey authorship) and TDXRay was added. All figures were redrawn.

Release boundary. The paper releases a specification, proofs under stated assumptions, and executable models. It does not release or claim a hardware design, firmware, or measured performance, and it makes no claim of peer review. The concepts were first shared publicly on 10 August 2026; this version is the archival release.

18. Conclusion

MECI reframes AI memory protection as a machine-enforced capability-separation problem. Its directly provable core is

$$\forall c \in \mathcal{C}_g : \text{Enabled}(c, s) \Rightarrow \neg \text{HighReach}(c, s), \quad (21)$$

where HighReach is reflexive-transitive influence in the unreleased state graph. A generation-checked, linearizable SafeOpen evaluated on the post-commit configuration lets cleanup remain multi-step and concurrent while preventing stale snapshots from enabling egress after high state reappears. Epoch revocation separately prevents reuse of old protected capabilities under the stated cryptographic assumptions.

Mutual exclusion still does not equal noninterference. The relational theorem requires explicit δ -release, low-stuttering high computation, a sound influence graph, ambient-channel equivalence, and a platform refinement; under those conditions the MECI invariant itself makes gated output depend only on low state,

and release-equivalent runs have equal modeled observation traces. The compositional claim is

$$\left. \begin{array}{l} \text{sound channel and influence model} \\ + \text{MECI gated-channel separation} \\ + \text{confined authorized declassification} \\ + \text{ambient-channel low equivalence} \\ + \text{fresh, non-resurrectable epochs} \\ + \text{implementation refinement} \end{array} \right\} \Rightarrow \text{noninterference modulo authorized release.} \quad (22)$$

The released artifacts make the abstraction falsifiable today. The next empirical milestone is an RMM or secure-monitor prototype with a platform refinement and measured transition costs. Until then, the paper keeps separate what is proved, what is exhaustively checked in a finite model, what is proposed, and what remains a hardware obligation.

Availability

The paper source, the TLA+ specification, the reference checker, and their captured outputs are published with this preprint and at <https://github.com/codethor0/meci>. Corrections, counterexamples, and implementations are welcome.

References

- [1] Z. Garby, A. D. Gordon, and D. Sands, “The LLMbda Calculus: AI Agents, Conversations, and Information Flow,” arXiv:2602.20064, 2026 (v2, July 2026).
- [2] R. Li, Z. Wang, Y. Yao, and X.-Y. Li, “MCP-ITP: An Automated Framework for Implicit Tool Poisoning in MCP,” arXiv:2601.07395, 2026.
- [3] Z. Zhou et al., “MCPShield: A Security Cognition Layer for Adaptive Trust Calibration in Model Context Protocol Agents,” arXiv:2602.14281, 2026.
- [4] J. A. Goguen and J. Meseguer, “Security Policies and Security Models,” in *IEEE Symposium on Security and Privacy*, pp. 11–20, 1982.
- [5] A. Sabelfeld and A. C. Myers, “Language-Based Information-Flow Security,” *IEEE Journal on Selected Areas in Communications*, vol. 21, no. 1, pp. 5–19, 2003.
- [6] A. C. Myers, A. Sabelfeld, and S. Zdancewic, “Enforcing Robust Declassification and Qualified Robustness,” *Journal of Computer Security*, vol. 14, no. 2, pp. 157–196, 2006.
- [7] O. Arden, A. Gollamudi, E. Cecchetti, S. Chong, and A. C. Myers, “A Calculus for Flow-Limited Authorization,” *Journal of Computer Security*, 2021; arXiv:2104.10379.
- [8] R. N. M. Watson et al., “Capability Hardware Enhanced RISC Instructions: CHERI Instruction-Set Architecture (Version 9),” University of Cambridge Computer Laboratory, Technical Report UCAM-CL-TR-987, 2023.
- [9] T. Murray et al., “seL4: From General Purpose to a Proof of Information Flow Enforcement,” in *IEEE Symposium on Security and Privacy*, 2013.
- [10] seL4 Foundation, “seL4 Proofs and Verification Assumptions,” project documentation, <https://sel4.systems>, accessed October 2026.
- [11] WebAssembly Community Group, “WASI: The WebAssembly System Interface, capability-based design documentation,” <https://github.com/WebAssembly/WASI>, accessed October 2026.
- [12] D. Zhang, Y. Wang, G. E. Suh, and A. C. Myers, “A Hardware Design Language for Timing-Sensitive Information-Flow Security,” in *ASPLOS*, 2015.

- [13] K. Cheang, “Formal Specification and Verification of Secure Information Flow for Hardware Platforms,” Ph.D. dissertation, Technical Report UCB/EECS-2023-224, University of California, Berkeley, 2023.
- [14] Arm Ltd., “Memory Encryption Contexts (MEC) for the Realm Management Extension,” Arm documentation and Learning Paths, accessed October 2026.
- [15] Intel Corporation, “Intel Trust Domain Extensions (Intel TDX) Module Base Architecture Specification and documentation,” accessed October 2026.
- [16] AMD, “SEV Secure Nested Paging Firmware ABI Specification,” Publication 56860, rev. 1.59, 26 August 2026.
- [17] E. Apsey et al., “Confidential Computing on NVIDIA H100 GPUs for Secure and Trustworthy AI,” NVIDIA Technical Blog, 3 August 2023, <https://developer.nvidia.com/blog/confidential-computing-on-h100-gpus-for-secure-and-trustworthy-ai/>.
- [18] Y. Yuan, Z. Liu, S. Deng, Y. Chen, S. Wang, Y. Zhang, and Z. Su, “CipherSteal: Stealing Input Data from TEE-Shielded Neural Networks with Ciphertext Side Channels,” in *IEEE Symposium on Security and Privacy*, pp. 4136–4154, 2025, doi:10.1109/SP61157.2025.00079.
- [19] J. Chuang, A. Seto, N. Berrios, S. van Schaik, C. Garman, and D. Genkin, “TEE.fail: Breaking Trusted Execution Environments via DDR5 Memory Bus Interposition,” in *IEEE Symposium on Security and Privacy*, 2026.
- [20] T. Hornetz, H. Yavarzadeh, A. Cheu, A. Gascón, L. Gerlach, D. Moghimi, P. Schoppmann, M. Schwarz, and R. Zhang, “TDXRay: Microarchitectural Side-Channel Analysis of Intel TDX for Real-World Workloads,” in *IEEE Symposium on Security and Privacy*, 2026, doi:10.1109/SP63933.2026.00170.
- [21] M. M. Hassan et al., “Memory Under Siege: A Comprehensive Survey of Side-Channel Attacks on Memory,” arXiv:2505.04896, 2025.
- [22] T. Thor, “Mission-Invariant Architecture Morphing: Service-Graph Reconfiguration Against Post-Access Reconnaissance, with Cryptographic Epoch Isolation and Mission-Domain State Continuity,” v1.0.0, Zenodo, 2026, doi:10.5281/zenodo.23001045.
- [23] T. Thor, “Attack Calculus: A Typed, Evidence-Aware State-Transition Calculus for Cross-Domain Cybersecurity Reasoning,” v1.0.0, Zenodo, 2026, doi:10.5281/zenodo.23092790.
- [24] Intel Corporation, “TEE.fail,” Intel Security Announcement 2025-10-28-001, October 2025, <https://www.intel.com/content/www/us/en/security-center/announcement/intel-security-announcement-2025-10-28-001.html>.
- [25] AMD, “Compromising Trusted Execution Environments through DDR5 Memory Bus Interposition,” AMD-SB-3040, 28 October 2025, <https://www.amd.com/en/resources/product-security/bulletin/amd-sb-3040.html>.
- [26] P. Rogaway, “Authenticated-Encryption with Associated-Data,” in *ACM Conference on Computer and Communications Security (CCS)*, pp. 98–107, 2002.
- [27] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*, Addison-Wesley, 2002.
- [28] TLA+ project, “TLA+ Tools (tla2tools.jar, TLC model checker),” release v1.7.4, <https://github.com/tlaplus/tlaplus>.
- [29] Arm Ltd., “Realm Management Extension and RME Device Assignment architecture documentation,” accessed October 2026.
- [30] Intel Corporation, “MKTME Side Channel Impact on Intel TDX,” Intel Software Security Guidance, accessed October 2026.
- [31] AMD, “AMD EPYC Processor Vulnerabilities, February 2026,” AMD-SB-3023, 2026.
- [32] NVIDIA, “Confidential Compute on NVIDIA Hopper H100,” NVIDIA whitepaper, 2023.

Appendix A. Reproducibility Manifest

SHA-256 digests of the released artifacts:

Artifact	SHA-256
scripts/meci_modelcheck.py	ade987c2591598ba4a06d7b538213ca328509e64da1c6d55cbfd15bad571711d
scripts/meci_modelcheck_results.txt	f26bba5985d1ddbda4146066497631bf206d738467f9ea3eabcc08e70538a1c
spec/MECI.tla	5cfd530b721d4b96ea1164902e54d5f359afc4633505a13f3783215b6008a45a
spec/MECI.cfg	db900c48e1378059063910c950b15744e4c5d93c93cbb957cdd117fb224396b2
spec/MECI_atomic.cfg	5e3fc1ee047c57e1f6b7c6b56850453c5bf01c43e5289a94aa7dbffd5b7065d3
spec/MECI_gencheck.cfg	db900c48e1378059063910c950b15744e4c5d93c93cbb957cdd117fb224396b2
spec/MECI_buggy.cfg	148e54e50a89d4df24fb381618b44728732a01d2f4d683a70c0ddafbd7aa54f7
spec/MECI_dponly.cfg	4126b634be7c5d54ed151a4673568a0a7484df5767be8743a3ad54cd5857049f
spec/MECI_noB.cfg	0416270218a2301cf00065924db6bddae3f0ca241fad36509476ec69838813ff
spec/tlc_results.txt	6b3ddaae939bf8e7ea7a834751b2bac5c2904ee535267b434e6a77cfa9934ecc

Appendix B. TLA+ Safety Specification Excerpt

```

D == epoch \notin revoked
Safe == hazards = {} /\ ~D
FreshPermit == permit /\ permitGen = gen

Produce(h) ==
  /\ "Q" \in hazards /\ h \notin hazards /\ gen < GenBound
  /\ hazards' = hazards \cup {h} /\ gen' = gen + 1
  /\ UNCHANGED <<egress, permit, permitGen, epoch, revoked>>

SafeOpen ==
  /\ ~egress /\ Safe /\ egress' = TRUE
  /\ UNCHANGED <<hazards, gen, permit, permitGen, epoch, revoked>>

OpenWithGeneration ==
  /\ ~egress /\ FreshPermit /\ egress' = TRUE /\ permit' = FALSE
  /\ UNCHANGED <<hazards, gen, permitGen, epoch, revoked>>

MECIInvariant == egress => Safe
PermitInvariant == FreshPermit => Safe
NoResurrection == [] [revoked \subseq revoked']_vars
THEOREM Spec => []MECIInvariant

```

The full module defines cleanup, revocation, producer restart, the three weakened open designs, and close-and-advance.

Appendix C. Platform Refinement Checklist

Before an implementation may inherit the abstract theorems, its evidence package should answer:

1. Which trusted component exclusively owns every gated egress-enable primitive?
2. How are direct and implicit influence edges conservatively derived from memory maps, capabilities, device queues, DMA mappings, tool handles, and control flow?
3. What events increment the MCB generation and invalidate an outstanding SafeOpen permit?
4. How are CPU, accelerator, device, DMA, and asynchronous software producers shown quiescent?

5. How are epoch and counter monotonicity and irreversible revocation preserved across reboot, migration, and recovery?
6. What exact code or hardware implements δ , and how is an attacker prevented from widening it?
7. How are sealed objects labeled, and how does the implementation track which keys can open them?
8. Which ambient channels are in scope, and what evidence establishes release-equivalent observations for each?
9. Which reset or partial-failure paths guarantee that egress remains closed?